



Introduzione ad Angular

Python

Cos'è Angular

Angular è un framework frontend basato su TypeScript, utilizzato per creare applicazioni web dinamiche e strutturate.

A differenza di una semplice pagina HTML, un'app Angular è composta da più parti organizzate che collaborano tra loro per gestire interfaccia, dati, eventi e navigazione.

Angular viene utilizzato soprattutto per applicazioni web medio-grandi, dashboard, gestionali, portali e Single Page Application.

Esempio minimo:

```
1. export class AppComponent {  
2.   titolo = "Hello Angular";  
3.}
```



Evoluzione di Angular

Angular nasce dall'evoluzione di AngularJS, framework pubblicato da Google nel 2010 e basato principalmente su JavaScript.

Con Angular 2, rilasciato nel 2016, il progetto viene ripensato introducendo TypeScript, un'architettura fortemente basata sui componenti, dependency injection e strumenti più adatti alla costruzione di applicazioni complesse.

Da quel momento Angular ha seguito un ciclo di aggiornamenti regolare, evolvendo verso un framework più modulare, performante e moderno; le versioni attuali includono funzionalità come standalone components, Signals e deferrable views, mantenendo però il componente come elemento fondamentale dell'architettura.



Angular oggi

Oggi Angular viene utilizzato soprattutto per costruire applicazioni web strutturate e scalabili, in particolare gestionali aziendali, dashboard, portali, applicazioni amministrative e grandi Single Page Application.

È particolarmente adatto a progetti sviluppati da team numerosi perché offre un ecosistema molto strutturato con soluzioni integrate per componenti, routing, form, dependency injection e gestione reattiva dello stato.

Angular è ancora mantenuto attivamente da Google e viene presentato ufficialmente come un framework pensato per applicazioni capaci di crescere sia in termini di codice sia di dimensione del team di sviluppo.



Angular utilizza principalmente TypeScript, un linguaggio basato su JavaScript che aggiunge tipizzazione, classi, interfacce e controlli utili durante lo sviluppo, ma per chi parte da zero, è importante capire che Angular non sostituisce JavaScript: utilizza TypeScript, che viene poi trasformato in JavaScript eseguibile dal browser.

```
1.nome: string = "Mario";  
2.eta: number = 25;
```

Rispetto a JavaScript:

```
1.let nome = "Mario";  
2.let eta = 25;
```



Con TypeScript possiamo dichiarare esplicitamente il tipo del dato.

Cos'è TypeScript

TypeScript è un linguaggio di programmazione sviluppato da Microsoft come estensione di JavaScript. Mantiene tutta la sintassi e le funzionalità di JavaScript, ma aggiunge strumenti come tipizzazione statica, interfacce, enum, generics e controlli sui tipi, che aiutano a individuare molti errori già durante la scrittura del codice. Il codice TypeScript non viene eseguito direttamente dal browser: viene prima compilato o transpilato in JavaScript, che può poi essere interpretato normalmente.

Rispetto a JavaScript, TypeScript rende il codice più prevedibile e manutenibile, soprattutto nei progetti di grandi dimensioni ed è per questo viene utilizzato fortemente in Angular, dove i dati possono sfruttare i tipi per migliorare la struttura dell'applicazione.



Angular è basato sui Component

Una delle caratteristiche fondamentali di Angular è l'utilizzo dei componenti.

Un componente rappresenta una parte dell'interfaccia e contiene normalmente logica e template.

Possiamo avere componenti separati per:

- Header
- Menu
- Login
- Prodotti
- Footer



Ogni componente può essere sviluppato e riutilizzato indipendentemente.

```
1. @Component({  
2.   selector: 'app-saluto',  
3.   template: '<h1>Ciao!</h1>',  
4. })  
5. export class SalutoComponent {  
6. }
```

selector identifica il componente, mentre **template** contiene ciò che deve essere visualizzato.

Nel codice HTML possiamo poi immaginarlo come:

```
1. <app-saluto></app-saluto>
```



Binding: collegare TypeScript e HTML

Angular permette di collegare direttamente i dati presenti nel codice TypeScript con l'interfaccia HTML. Questo meccanismo viene chiamato data binding.

Una delle forme più semplici è l'interpolazione:

```
1. export class AppComponent { nome = "Luca"; }
```

Nel template HTML:

```
1. <h1>Ciao {{ nome }}</h1>
```

Il browser mostrerà:

```
1. Ciao Luca
```

Angular mantiene quindi separati:

TypeScript → dati e logica E HTML → interfaccia
ma permette ai due livelli di comunicare facilmente.



Gestione degli eventi

Angular permette di reagire agli eventi generati dall'utente, come click, input da tastiera o modifica di un campo.

Nel componente creiamo il metodo: Nel template:

```
1. export class AppComponent {  
2.   messaggio = "";  
3.   saluta() {  
4.     this.messaggio = "Ciao!";}}
```

```
1. <button (click)="saluta()">  
2.   Premi  
3. </button>  
4. <p>{{ messaggio }}</p>
```



Quando l'utente preme il pulsante, Angular esegue il metodo saluta() e aggiorna automaticamente la pagina.

Hello World completo in Angular

Un primo programma Angular può essere composto da un componente che contiene un dato, un metodo e un template HTML.

TypeScript:

```
1. export class AppComponent {  
2.   titolo = "Hello Angular";  
3.   saluta() { this.titolo = "Benvenuto in Angular!"; }}
```

Html:

```
1. <h1>{{ titolo }}</h1>  
2. <button (click)="saluta()">  
3.   Cambia messaggio  
4. </button>
```



All'avvio viene visualizzato: Hello Angular
Quando premiamo il pulsante: Benvenuto in Angular!

Questo piccolo esempio contiene già le principali caratteristiche di Angular:

- **Component → AppComponent rappresenta una parte dell'applicazione.**
- **TypeScript → contiene dati e logica.**
- **Template HTML → definisce ciò che vede l'utente.**
- **Interpolation → {{ titolo }} visualizza un dato TypeScript.**
- **Event Binding → (click) intercetta il click dell'utente.**
- **Aggiornamento automatico della UI → modificando titolo, Angular aggiorna anche la pagina.**



Buon Davante a tutti

